

Code:

```
class FixedSizeStack:

    def __init__(self, capacity=4):

        self.stack = [None] * capacity

        self.top = -1

        self.capacity = capacity


    def push(self, value):

        if self.isFull():

            print("Stack is full. Cannot push:", value)

        else:

            self.top += 1

            self.stack[self.top] = value


    def pop(self):

        if self.isEmpty():

            print("Stack is empty. Cannot pop.")

            return None

        else:

            value = self.stack[self.top]

            self.stack[self.top] = None

            self.top -= 1

            return value


    def peek(self):
```

```
if self.isEmpty():  
    return None  
else:  
    return self.stack[self.top]
```

```
def isEmpty(self):  
    return self.top == -1
```

```
def isFull(self):  
    return self.top == self.capacity - 1
```

```
def print_stack(self, label="Stack"):  
    # For printing only the filled portion  
    values = [str(self.stack[i]) for i in range(self.top + 1)]  
    print(f"{label}: {' '.join(values)}")
```

```
stack = FixedSizeStack()
```

```
stack.push(10)
```

```
stack.push(20)
```

```
stack.push(30)
```

```
stack.print_stack("Stack")
```

```
stack.pop()
```

```
stack.print_stack("After Pop")
```

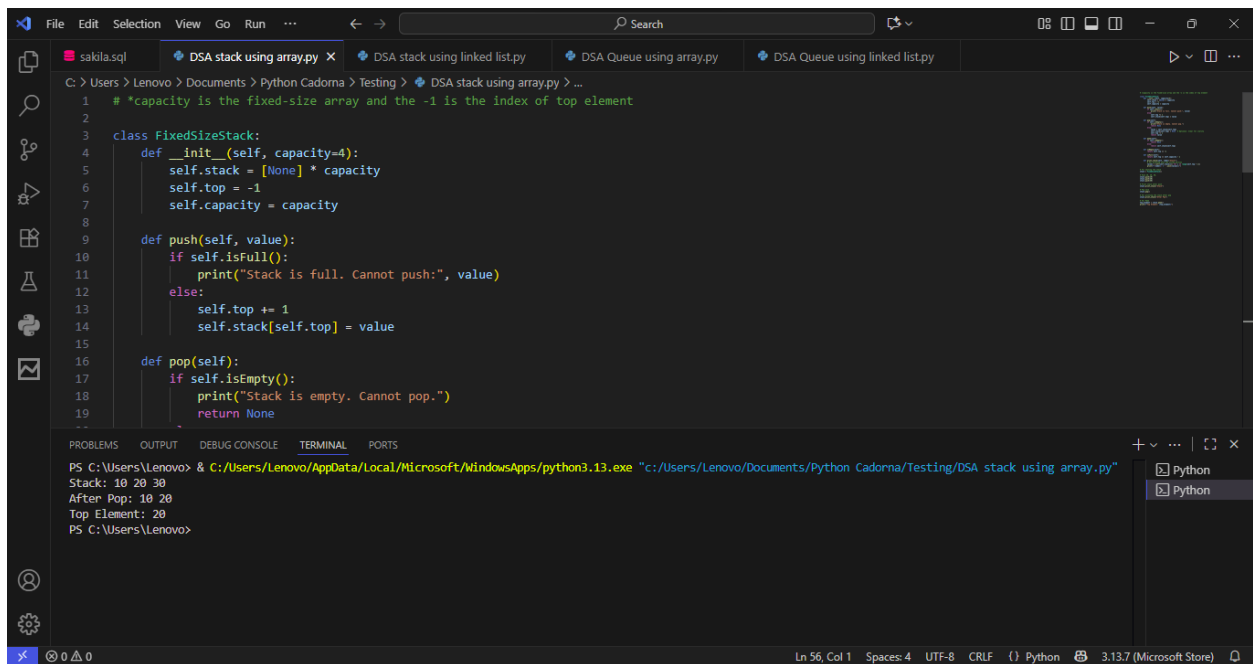
```
top_element = stack.peak()
```

```
print(f"Top Element: {top_element}")
```

Programiz Link:

<https://www.programiz.com/online-compiler/1S5r4IUf56CYn>

Sample Output:



The screenshot shows a code editor with a Python file named 'DSA stack using array.py'. The code defines a 'FixedSizeStack' class with methods for push, pop, and printing the stack. The terminal output shows the stack being pushed with values 10, 20, and 30, then popped, resulting in the top element being 20.

```
1 # *capacity is the fixed-size array and the -1 is the index of top element
2
3 class FixedSizeStack:
4     def __init__(self, capacity=4):
5         self.stack = [None] * capacity
6         self.top = -1
7         self.capacity = capacity
8
9     def push(self, value):
10        if self.isFull():
11            print("Stack is full. Cannot push:", value)
12        else:
13            self.top += 1
14            self.stack[self.top] = value
15
16    def pop(self):
17        if self.isEmpty():
18            print("Stack is empty. Cannot pop.")
19        return None
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\Users\Lenovo> & c:/Users/Lenovo/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Lenovo/Documents/Python Cadorna/Testing/DSA stack using array.py"
Stack: 10 20 30
After Pop: 10 20
Top Element: 20
PS C:\Users\Lenovo>
```